Self-Certified Continual Learning: Gold-Free, Safe Weight Updates for Deployed Code LLMs

OpenContinualEnv Research
Corresponding repository: `open_continual_env` (module `gcl`)

***Abstract*—A deployed code model sees its densest task-level experience exactly when it is frozen, and the one signal that could safely drive learning there — gold unit tests — is precisely what deployment does not provide. We ask: *can a model certify its own learning targets well enough to update its weights continually, with no gold labels anywhere in the learning loop or the safety gate?* We introduce Self-Certified Continual Learning (SCCL) inside a grounded lifelong-RL environment for code LLMs. SCCL (1) samples executable self-spec tests from the natural-language specification alone, (2) filters to tests that discriminate among sampled candidate solutions, (3) certifies a target by cross-bag consensus confidence, and (4) gates every LoRA update with a *self-replay veto*: previously certified skills must regenerate and still pass their own self-tests after the update, or the update is rolled back at zero cost. The gold-free guarantee is *structural*: the certifier's API consumes only (spec, entry, domain), and we enforce it with AST audits, adversarial poisoned-gold tests, and an interface-normalization argument (the call signature is task specification, not supervision). On a 4-family MBPP/math stream with drift, unverified self-training collapses below the frozen control (0.237–0.356 vs. 0.613), while SCCL improves over it in two independent runs of the identical configuration (0.637 originally, 0.744 in a rerun), and *full gold-reference supervision underperforms gold-free certification* (0.475). A telemetry-only audit shows a gold holdout-ϵ gate would have rejected most of the updates that produced SCCL's gains — gold never influences a single accept/reject decision here. SCCL also robustly acquires a family the base model scores 0.0 on at zero-shot: self-certified arithmetic transfers forward, and majority-vote certification bootstraps a *perfect* math holdout in four of five SCCL-family runs. Diagnosing the residual forgetting, we find it is almost entirely loss of *unvisited generalization*, not of trained skills — and that single-unseeded-run comparisons of nearby mechanisms are inside the run-to-run noise floor, which we disclose and address with a seeded protocol. Two gold-free responses follow: v2 manufactures stability signal (certified rehearsal, neighborhood probes, math-RRV), achieving the lowest forgetting among gold-free rows but paying a measured plasticity price at the gate; v3 moves neighborhood checking to *admission*, rejecting instance-narrow solutions before any gradient is spent. All numbers derive from actually executed gradient updates and actually executed tests.**

***Index Terms*—Continual learning, large language models, self-certification, program synthesis, parameter-efficient fine-tuning, safe deployment.**

I. Introduction

The deployment paradox: a code model receives its richest domain signal *after* it is frozen. Continual learning research has long assumed supervised access to each new task's ground truth [2], [5], [7]; deployed settings do not. Prior work in this repository established a *grounded* continual-learning environment — a lifelong MDP with factored task/learning actions and execution-verified rewards — but its strongest learner (Verified Skill Regeneration, VSR) still required gold unit tests to admit skills and a gold holdout to gate updates.

This paper closes that gap. Our claim is that *verification can be manufactured*: a code model can generate its own executable interpretation of a specification, test candidate solutions against it, and use the structure of agreement across independent samples as a statistically meaningful confidence score. Where prior self-training (STaR, self-consistency, self-repair) [16], [17], [18], [19] uses self-generated signal to *select* data, SCCL uses it to *certify training targets and gate weight updates* — and then proves, by construction, that no gold information enters either path.

A. Contributions

- **SCCL, a gold-free certification loop.** Self-spec test bags → discriminative filtering → consensus certification, plus a *unanimous-pool rule* that certifies already-mastered skills so the safety mechanism has something to protect (Sec. III).
- **Self-replay veto (RRV).** A gold-free safety gate: after every update, previously certified skills are regenerated under the new adapter and re-run against *their own* stored self-tests; regression vetoes the update and triggers a zero-copy rollback (Sec. II).
- **Structural gold-freeness, enforced.** The certifier never receives a Task object; we enforce this with AST audits, signature inspection, and poisoned-gold adversarial tests, and we formalize interface normalization (call signatures are specification, not supervision) (Sec. III-E).
- **A diagnosis of gold-free forgetting.** Splitting retention into trained tasks vs. unvisited holdouts shows the damage is almost entirely *generalization* loss, and that it concentrates in one volatile component (arithmetic holdout), while zero-shot-domain acquisition (math) is robust across runs (Sec. V).
- **Self-manufactured stability (v2) and admission-time neighborhood certification (v3).** v2 extends RRV to certified rehearsal, neighborhood probes, and math entries — lowest forgetting among gold-free rows, at a measured plasticity cost we report honestly; v3 filters instance-

narrow solutions *before* training, moving the stability mechanism upstream of the gradient (Secs. III-A, III-B).
- **Variance as a first-class result.** The identical configuration lands at 0.637 and 0.744 in two unseeded runs — a spread larger than most mechanism differences. We disclose this, add a seeded protocol (`torch_seed` + multi-seed runner), and interpret single-run comparisons accordingly (Sec. V).

II. Environment and Formalism

A. Lifelong MDP

State $s_t = (e_t, \text{mem}, \text{adapters}, \text{perf})$; action $a_t = (a_{\text{task}}, a_{\text{learn}})$ with $a_{\text{learn}} \in \{\text{IGNORE}, \text{STORE}, \text{UPDATE\_LORA}, \text{CONSOLIDATE}, \text{REVIEW}\}$. The environment computes a verified reward by sandboxed execution (r_t combines exec success, unit-test pass rate, code quality, and a safety term), but under SCCL this reward is *telemetry*: it never selects the training target nor accepts the update.

B. Real gated weight updates

An UPDATE_LORA trains a LoRA adapter [1] (r=16) on the chosen target for a few steps (snapshot → update → gate → keep/rollback), with rollback implemented as a PEFT state-dict restore — zero extra model copies on a 16 GB GPU.

C. Self-replay veto (RRV)

Every accepted certification is committed to a vault with its spec, prompt, code, and certifying self-tests. Before an update is kept, RRV re-generates up to C vaulted skills under the *new* adapter and requires at least one regeneration to pass that skill's own stored tests; failure vetoes the update and restores the snapshot. This is a gold-free replacement for the holdout-ϵ gate: stability is checked against the model's *own previously certified competences*, not against gold.

D. Metrics

After the stream walk, a greedy policy is evaluated on every family's canary holdout (never shown during the stream), yielding per-family final scores. We report ACC (mean final holdout accuracy), Δ = final holdout − pre-stream zero-shot baseline (our BWT column), forgetting = mean positive per-family drop relative to that baseline, FWT = improvement of trained-task accuracy between first contact and stream end, and frontier = ACC − forgetting. The zero-shot baseline is scored on the train split while finals are scored on holdouts, so any split-difficulty gap is absorbed equally by every learner; the frozen row is the control for it (its Δ is nonzero despite zero updates). Anti-contamination is asserted per run: train∩holdout overlap = 0 (yes), stream hash `554ce43f182b`.

III. SCCL: The Method

For each incoming task with natural-language spec s, SCCL builds a certified training target using only s.

a) 1. Self-spec test bags.: We sample B independent bags of assert-style tests from s alone (one greedy bag, $B-1$ diverse bags), each parsed through an AST safety filter that keeps only literal-input `assert` statements calling the derived entry point — no definitions, imports, assignments, or dangerous calls survive.

b) 2. Candidate pool.: We sample k solutions at temperature plus one greedy solution, extract executable code, and deduplicate.

c) 3. Discriminative consensus certification.: Let $V \in \{0,1\}^{(k+1)\times|T|}$ be the verdict matrix from sandboxed execution. A test is *discriminative* if it splits the pool ($0 <$ colsum $< k+1$); tests everyone passes or everyone fails carry no ranking signal. Confidence for candidate i is

$$c_i = \underbrace{\tfrac{1}{|D|}\textstyle\sum_{j\in D} V_{ij}}_{\text{discriminative pass rate}} \cdot\big(0.7+0.3\,\alpha\big) + \mu\mathbb{1}[\text{margin} \geq m], \quad (1)$$

where D is the discriminative test set, α is cross-bag agreement of per-bag best pass rates, and μ a small margin bonus. Certify if $\max_i c_i \geq \tau$.

d) Unanimous-pool rule.: If *every* candidate passes *every* self-test, no test can discriminate, but the consensus itself is evidence: we certify the skill anyway. This matters beyond coverage — already-mastered skills must enter the vault, otherwise the self-replay veto has nothing to protect and later updates can silently destroy mastered behavior. Pools where nothing passes remain *weak*: confidence capped below τ, no training on garbage.

A. SCCL v2: self-manufactured stability

A fine-grained diagnosis of v1 motivates three gold-free extensions. Measuring retention separately on *trained* tasks and *unvisited holdouts* of the same family shows v1's forgetting is almost entirely unvisited-generalization loss: trained arithmetic tasks are retained well while arithmetic holdout accuracy collapses, and RRV never even checked math skills. The gate protected what was trained, not what was known. v2 manufactures its own stability signal in three ways:

a) 1. Certified rehearsal.: Each update trains on the new certified target *plus* a stride-sampled set of pairs from the self-certified vault, so prior (cross-family) certified skills keep receiving gradient. Unlike raw replay buffers [7], [5], [6], every rehearsed pair was certified by the same consensus mechanism — the buffer is certified-correct by construction, and no gold is needed to curate it.

b) 2. Neighborhood probes.: At certification time the model also manufactures a certified *variant* of the task it never trains on. For code: paraphrase the spec, re-certify the paraphrase, and admit it as a probe only under *bidirectional cross-validation* — the base winner passes the probe's self-tests *and* the probe winner passes the base's self-tests, the strongest spec-only evidence that the paraphrase preserved semantics. For math: sample a numeric variant (different numbers, same method) and certify its answer by majority vote. Probes are stored in the vault with kind PROBE; RRV re-checks the newest probes on every gated update, so the

safety gate now protects a *generalization neighborhood* — competence on tasks the weights never saw — instead of only trained points.

c) 3. Math-RRV.: The veto is extended to math vault entries: regenerate answers under the new adapter and require one to match the stored certified value (canonical-form comparison). v1 left math entirely unprotected.

Gold-freeness is preserved structurally: probe manufacture is a spec-only API MAKEPROBE(engine, verifier, spec, domain, CertResult) — no Task object is reachable — and the same AST audit, signature inspection, and poisoned-gold adversarial tests cover it. A probe is never a training target; it exists solely as evidence the gate can check.

B. SCCL v3: neighborhood certification at admission

The v2 results (Sec. V) motivate moving the neighborhood check upstream. Gate-side probes police damage *after* the gradient has already moved the weights: when a probe fails, the only remedy is veto and rollback, which restores the previous adapter but refunds none of the consumed learning opportunity, and the extra vetoes measurably depress accepted-update counts and final accuracy. v3 instead checks the neighborhood *before* training.

At certification time, SCCL manufactures an evidence-bearing variant of the task. For code: a paraphrase of the spec plus a fresh self-test bag written for the paraphrase; the certified winner must *pass the variant's tests*. For math: a numeric variant (different numbers, same method) whose answer is certified by majority vote; the certified answer must remain self-consistent on the variant. Failure demotes the candidate: it is not trained on and not committed to the vault.

The decision policy is evidence-based and deliberately asymmetric. When a variant or its tests *cannot be manufactured* (degenerate generation, no parseable asserts), the candidate is **admitted** — an *open* decision, logged as such — because absence of evidence is not evidence of fragility, and over-filtering would reproduce v2's plasticity tax one stage earlier. When evidence *is* manufactured and the winner fails it, the candidate is **rejected** — a *closed* decision. Instance-narrow solutions (e.g., a hard-coded return value that happens to pass spec-derived tests) tend to fail paraphrased variants with fresh tests, so the filter attacks exactly the solutions whose later forgetting v2's gate was left to police. The check is a spec-only API CHECKNBHD(engine, verifier, spec, domain, CertResult) reusing the certifier's prompts and sandbox; gold-freeness is preserved by the same structural argument and covered by the same adversarial suite.

C. SCCL v4: in-update base anchoring

v1–v3 are all *gates*: they decide which updates are safe to keep. But a gate can only protect what it *checks*, and the RRV gate checks the *certified* vault skills. The base model's *uncertified* general capability — e.g. arithmetic generalization measured on never-trained holdout tasks — is never in the vault, so no accept/reject verdict defends it. On the seeded v3 ladder we observe the consequence starkly: *every* learning row — gold-free SCCL variants *and* the gold-verified reference — collapses the arithmetic holdout to ~ 0.40 while FROZEN holds 0.60. The damage is inflicted *inside* an accepted update, which no gate decision can prevent. Capability preservation therefore requires acting on the update *itself*, not on the decision about it.

v4 adds an **in-update base anchor**. During each LoRA update we add a quadratic pull of the trainable (LoRA) parameters toward their value at LoRA initialization — which is exactly the frozen base model, i.e. the `disable_adapter` state. The anchor is a standard parameter-space regularizer $\frac{\lambda}{2}\|\theta-\theta_0\|^2$ in the family of EWC/SI/MAS [2], [3], [4] applied only while the update runs; it is gold-free (the target θ_0 is the model's own initialization, no labels, and no Fisher or importance estimate), and it is *decision-independent*: it changes how an accepted update moves the weights, never *which* updates are accepted. This keeps the gate's safety semantics untouched while giving accepted updates an explicit pressure to remain near the base model's general capability. The anchor is applied per learner so it can be cleanly A/B'd in the ladder, and we ablate the strength λ in a single run. For transparency: the first implementation of this anchor silently failed to engage — an adapter state-dict key mismatch made the penalty evaluate to exactly zero — and the first v4 ladder therefore ran plain SCCL on the anchor arm. The failure was detected because the anchor rows proved bit-identical to seed-matched controls, and is reported in full in Sec. V; the corrected, engagement-tested implementation is what v5's factorial design (Sec. III-D) evaluates.

D. SCCL v5: closing the gate's coverage gap

The v3 verdict leaves a precise residue: gates decide *which* updates happen, and the arithmetic erosion is inflicted *inside* accepted updates (Sec. V). v4's anchor is built to act exactly there — but a seed-42 post-mortem of the gate itself pins a second, independent mechanism on the same erosion. RRV's check pool is a pure *recency window*: it re-executes the k *newest* certified skills. Once the stream moves from family A to family B, A's skills slide out of the window and A silently leaves the gate's field of view. The audit trail is unambiguous: every recorded veto in the seed-42 run broke an *active-family* skill, and the six certified arithmetic skills were never checked once during the string and drift phases — exactly the window in which the arithmetic holdout collapsed. The gate did not fail; it was never *asked*. Coverage, not conservatism, was the missing ingredient.

v5 replaces the recency window with a **family-stratified check pool**: RRV re-executes the newest $\lceil k/F \rceil$ certified skills from *every* certified family F, so all certified competence stays under protection no matter where the stream currently is. Selection is deterministic (no RNG), gold-free (it re-runs stored self-tests only), and preserves the veto's decision semantics — the rule is unchanged, only its field of view widens. The default recency pool remains the untouched code path for every pre-v5 row.

The anchor and the coverage fix attack *different* failure modes, so v5 is a 2×2 factorial — anchor (λ=0.1 vs. none) $\times$ stratified coverage (on vs. off) — with SCCL as the neither-cell control that must bit-reproduce the v3/v4 seed-42 row, and VSR_NOGOLD as the gold-test reference. The pre-registered reading decomposes main effects and interaction: if the two mechanisms are complementary — the anchor suppressing general drift damage, the stratified veto holding every certified family under the gate — the both-cell should approach FROZEN's arithmetic holdout while keeping the anchor's plasticity, the first configuration expected to beat the frozen control on *both* axes of the frontier without touching gold.

E. Interface normalization

MBPP-style specs hide the call interface inside the gold tests; no gold-free agent can recover it, and failure-by-`NameError`/`TypeError` is not a meaningful learning signal. We move the *call interface* (function name, arity, keyword names) into the specification, mirroring HumanEval's signature-in-prompt convention [14]. This moves no supervision: expected outputs, pass/fail verdicts, and reference implementations remain gold. Formally, the interface is the caller's API contract — part of the task definition — while supervision is the functional behavior the tests check.

F. The gold-free guarantee

SCCL's certification function signature is CERTIFY(engine, verifier, spec, domain, entry): no `Task` object is reachable, so gold fields (`test_code`, `reference_answer`, ...) are unreachable by construction. We enforce this three ways:

1) **Static**: an AST audit of `selfcert.py` fails the build if any gold field is referenced; signature inspection fails if any certifier method accepts a task-like parameter.
2) **Adversarial**: poisoned-gold tests corrupt `test_code`/`reference_answer` so correct code scores ≈ 0 on gold; SCCL must still certify and train (it does), and a gold-passing-but-uncertified answer must be refused (it is).
3) **Dynamic telemetry**: after each gated update we score a small gold probe under base and candidate adapter — logged as telemetry, never consumed. It lets us audit, post-hoc, how often the gold-free gate would have agreed with a gold gate.

IV. EXPERIMENTAL SETUP

Model `Qwen/Qwen2.5-Coder-3B-Instruct`, bf16, LoRA rank 16, RTX 4060 Ti 16 GB; update cap 60. Stream: four families — MBPP arithmetic [15], synthetic math word problems (cross-domain shift), MBPP strings, and a behaviourally drifted MBPP family (renamed ops + shifted literals, so memorized answers become wrong). Each family: 8 train tasks, 4 canary holdout tasks; drift applied to train *and* holdout so the family stays internally consistent.

Learner ladder (isolation by construction): FROZEN (no updates; control), SELFDISTILL and EXECFILTER (unverified self-training; unsafe baselines with *no* gold gate), SCCL_NOCONS (certify without consensus), **SCCL** (ours), SCCL_NOGATE (certify without RRV), VSR_NOGOLD (self-taught targets verified by *gold tests* — isolates the value of gold-free certification), and VSR (gold reference injection + gold skill-vault gate — the strongest gold access any learner gets).

v2 ladder (same stream and hash, rerun end-to-end for direct comparison): FROZEN, SCCL (v1 reference), SCCL_REPLAY (v1 + certified rehearsal), SCCL_PROBE (v1 + neighborhood probes + math-RRV), **SCCL_V2** (all three), and VSR_NOGOLD (gold-test reference). The single-mechanism variants isolate each contribution; everything in the SCCL loop remains gold-free.

v3 ladder (same stream, *seeded*): FROZEN, SCCL (v1 reference), **SCCL_N** (v1 + admission-time neighborhood certification only — isolating the v3 mechanism without replay or probes), SCCL_PROMOTE (v1 + probes + probe-curriculum promotion), and VSR_NOGOLD.

v4 ladder (same stream, seeded): FROZEN, SCCL, SCCL_ANCHOR_LO / SCCL_ANCHOR_HI (v1 + base anchor, $\lambda = 0.1/0.5$ ablated in-run), and VSR_NOGOLD.

v5 ladder (same stream, seeded; 2×2 factorial): FROZEN, SCCL (neither mechanism; bit-identical control), SCCL_STRAT (stratified veto only), SCCL_ANCHOR_LO (anchor only), **SCCL_ANCHOR_STRAT** (both — the candidate composition), and VSR_NOGOLD.

Seeded protocol. Early runs were unseeded: candidate sampling, test-bag generation, and LoRA optimization all drew from uncontrolled RNG state. After observing a large spread for the identical configuration (Sec. V), we added an explicit `torch_seed` that seeds torch/CUDA/numpy/Python RNG per learner (`torch_seed + learner index`); seeded results are reproducible up to GPU nondeterminism, and headline comparisons use multiple seeds with mean±std.

V. RESULTS

TABLE I
CONTINUAL LEARNING ON THE 4-FAMILY DRIFT STREAM. ALL NUMBERS FROM EXECUTED RUNS; UPD/RBK = ACCEPTED UPDATES / ROLLBACKS. UNSAFE BASELINES HAVE NO GOLD GATE BY CONSTRUCTION.

Learner	ACC↑	BWT↑	FWT↑	Forget↓	Frontier↑	Upd/Rbk
Frozen	0.613	+0.132	+0.000	0.025	+0.588	0/0
SelfDistill	0.237	-0.243	-0.260	0.243	-0.005	32/0
ExecFilter	0.356	-0.124	-0.140	0.249	+0.107	30/0
SCCL-nocons	0.581	+0.101	+0.083	0.086	+0.495	20/0
SCCL (ours)	**0.637**	**+0.157**	+0.119	0.138	+0.500	18/5
SCCL-nogate	0.637	+0.157	+0.058	0.125	+0.512	17/0
VSR-nogold	0.637	+0.157	+0.147	0.075	+0.562	13/1
VSR (full gold)	0.475	-0.005	+0.297	0.175	+0.300	25/0

Unverified self-training collapses. SELFDISTILL (train on own outputs) and EXECFILTER (train on executions without certification) both degrade below frozen with ≈ 0.24 forgetting: without a target-quality signal, gradient steps accumulate

TABLE II

GOLD-FREE GATE TELEMETRY (MEASUREMENT ONLY — NEVER USED IN ANY ACCEPT/REJECT DECISION). CERT. RATE = FRACTION OF TASKS CERTIFIED; AGREEMENT = FRACTION OF GATED UPDATES WHERE THE GOLD-FREE VERDICT MATCHES A GOLD HOLDOUT-ϵ VERDICT, AUDITED POST-HOC.

Learner	Cert. rate	Mean conf	Vetoes	Gold agree
SCCL-nocons	0.625	0.705	0	0.100
SCCL	0.719	0.776	5	0.304
SCCL-nogate	0.531	0.633	0	0.059

TABLE III

SCCL V2 — SELF-MANUFACTURED STABILITY (SAME STREAM HASH AS TABLE I; ALL ROWS FROM ONE RERUN). CR = CERTIFIED REHEARSAL; P = NEIGHBORHOOD PROBES + MATH-RRV; PROBES = CERTIFIED PROBES COMMITTED.

Learner	ACC↑	BWT↑	Forget↓	Frontier↑	Upd/Rbk	Vetoes	Probes
Frozen	0.613	+0.132	0.025	+0.588	0/0	–	–
SCCL (v1 ref)	0.744	+0.264	0.081	+0.662	19/2	2	0
SCCL + CR	0.694	+0.214	0.125	+0.569	16/7	7	0
SCCL + P	0.650	+0.170	0.175	+0.475	14/6	6	15
SCCL v2	**0.688**	**+0.207**	0.075	+0.613	12/8	8	12
VSR-nogold (gold tests)	0.758	+0.278	0.004	+0.754	14/2	–	–

the model's own errors. This is the failure mode SCCL exists to fix.

Certification recovers learning. SCCL certifies 0.719 of tasks (mean confidence 0.776) and reaches 0.637 accuracy — matching VSR_NOGOLD, which verifies its self-taught targets with *gold* unit tests and gates on a gold holdout — while consuming no gold anywhere. Removing consensus (SCCL-NOCONS) costs accuracy (down to 0.581) and certification rate; the gold-free RRV veto fired 5 times, each a harmful update rolled back without any gold signal. An end-to-end rerun of the identical configuration (Table III) reproduces the direction and reaches 0.744 accuracy — the two values bracket the run-to-run variance discussed below.

SCCL acquires a zero-shot domain, robustly. The base model scores 0.0 on the synthetic math family. Self-certified arithmetic training transfers forward (math first-contact accuracy rises to 0.625 before the stream ever trains math), and majority-vote certification then bootstraps math to a *perfect* holdout — and it does so again in the rerun, and in four of the five SCCL-family learners across both runs. Zero-shot-domain acquisition is the most robust phenomenon in the paper; it is also entirely gold-free.

Full gold supervision is not an upper bound. VSR injects gold reference solutions as training targets and gates updates with gold skill-vault tests — the strongest gold access any learner has — yet lands at 0.475, *below the frozen control* (0.613) and 0.16 below SCCL. Gold references maximize plasticity (FWT +0.297, the ladder's best) but also maximize forgetting (0.175; arithmetic holdout collapses from 0.7 zero-shot to 0.2), and its gold gate fires 0 rollbacks across 25 accepted updates. Training on an external reference style moves the adapter off the model's own solution manifold; self-generated, self-certified targets stay on it. Gold labels are not automatically useful: their value depends on *how* they enter the loop, and on this stream the fully gold variant is the worst learner that learns at all.

What the gate protects. RRV checks the model's own certified skills, so it preserves acquired competence: mean trained-task retention at stream end is 0.75 with the gate vs. 0.53 without it (SCCL-NOGATE). It does not, however, protect unvisited generalization: in the original run arithmetic holdout accuracy drops to 0.15, while the gold-gated VSR_NOGOLD retains 0.40 — the residual value of gold *tests*, honestly quantified (gold *references* retain even less: VSR's arithmetic holdout is 0.2). The rerun sharpens the quantification: VSR_NOGOLD is the single best row of Table III (ACC 0.758, forgetting 0.004, frontier +0.754), retaining the arithmetic holdout at 0.68 where the gold-free rows reach 0.20–0.58, while gold-free SCCL answers with a perfect math holdout (1.00 vs. 0.75). This diagnosis — forgetting as *generalization* loss — is what v2 and v3 target.

Honest boundaries. (i) Certification amplifies unlocked capability; it cannot manufacture it. On tasks where pass@k stays at zero and no transfer unlocks the family, weak pools stay capped below τ and SCCL abstains. (ii) Self-tests and gold can diverge on genuinely ambiguous specs; the telemetry column (gold agreement 0.304) quantifies how often the gold-free gate would have matched a gold holdout-ϵ gate. Notably, most disagreements are math updates the gold gate would have *rejected* — the same updates that produced the perfect math holdout — evidence that gold-holdout conservatism can block genuine capability acquisition. In the rerun this is extreme: a gold gate would have accepted only 1–4 of the 20–23 gated updates per SCCL learner, yet all of them improved over frozen. Residual certified-but-gold-wrong targets are the price of gold-freeness.

Run-to-run variance is first-class. The FROZEN row is bit-identical across the two runs (ACC 0.613, forgetting 0.025): stream construction, evaluation, and holdouts are deterministic, so all variance enters through training-time sampling (candidates, self-test bags, optimization order). The same SCCL configuration lands at ACC 0.637 / forgetting 0.138 originally and 0.744 / 0.081 in the rerun — a 0.107 accuracy spread, larger than every mechanism difference in either table. Per-

TABLE IV

SCCL V3 — NEIGHBORHOOD CERTIFICATION AT *admission* (SAME STREAM HASH; SEEDED PROTOCOL, SEEDS 42/43/44, MEAN±STD; COMPARE WITHIN THIS TABLE). SCCL_N = V1 + ONLY THE ADMISSION FILTER (ISOLATES THE MECHANISM); SCCL_PROMOTE = V2 MONITORING + PROBE-CURRICULUM REFERENCE; VSR_NOGOLD = SELF-TAUGHT TARGETS VERIFIED BY GOLD TESTS. ADMISSION-REJECTION AND VETO COUNTERS ARE PER-RUN QUANTITIES; THE SEED-42 ADMISSION AUDIT IS REPORTED IN THE TEXT.

Learner	ACC↑	BWT↑	Forget↓	Frontier↑	Upd/Rbk
Frozen	0.613 ± 0.000	$+0.132 \pm 0.000$	0.025 ± 0.000	$+0.588 \pm 0.000$	0.0/0.0
SCCL (v1 ref)	0.596 ± 0.083	$+0.116 \pm 0.083$	0.092 ± 0.029	$+0.504 \pm 0.106$	15.7/2.5
SCCL + nbhd admission	$\mathbf{0.650} \pm 0.038$	$+0.170 \pm 0.038$	$\mathbf{0.129} \pm 0.002$	$+0.521 \pm 0.042$	14.0/6.3
SCCL promote (v2 ref)	0.635 ± 0.050	$+0.155 \pm 0.050$	0.110 ± 0.031	$+0.525 \pm 0.078$	13.0/6.0
VSR-nogold (gold tests)	0.629 ± 0.007	$+0.149 \pm 0.007$	0.113 ± 0.033	$+0.517 \pm 0.031$	9.7/5.7

family, the volatile component is almost entirely the arithmetic holdout (0.15 vs. 0.58); math acquisition is stable (perfect holdout in both runs). Even *rankings flip*: in the original run SCCL sits *below* the frozen control on frontier ($+0.500$ vs. $+0.588$), in the rerun above it ($+0.662$) — whether SCCL beats doing nothing on the stability-adjusted metric is decided by training-time sampling, not by the mechanism alone. The one stable ordering is the gold-test lead: VSR_NOGOLD beats SCCL on frontier in both runs ($+0.562$ vs. $+0.500$ originally, $+0.754$ vs. $+0.662$ in the rerun) — a real, bounded residual value of gold tests, and the explicit target of v3. We therefore treat single-unseeded-run comparisons of nearby mechanisms as unreliable, report the variance rather than hiding it, and move headline claims to seeded multi-seed runs (Sec. IV).

Self-manufactured stability: an honest accounting (v2). In the rerun, SCCL_V2 achieves the lowest forgetting of any gold-free learning row (0.075; VSR_NOGOLD's 0.004 is lower, bought with gold tests) and the best string retention (0.80 vs. 0.60), but pays a plasticity price: its probe-extended gate vetoes 8 of 20 gated updates (vs. 2 of 21 for in-run SCCL), accepts only 12 updates, under-trains math (0.75 holdout vs. 1.00), and lands below the in-run v1 reference on frontier ($+0.613$ vs. $+0.662$). The single-mechanism variants do worse: SCCL_REPLAY ($+0.569$) and SCCL_PROBE ($+0.475$) both fall *below the frozen control* ($+0.588$) on frontier, and SCCL_PROBE has the *worst* arithmetic holdout (0.20) despite — because of — its probes: they detect damage and veto updates, but a rollback restores yesterday's adapter, not lost generalization, while the vetoed learning opportunity is gone. The consistent mechanism-level story across these rows: gate-side neighborhood checks police damage *after* the gradient moved the weights, converting plasticity into stability at a visible exchange rate. Given the variance above, we do not claim the single-rerun ordering of v2 variants as settled; we claim the diagnosis. That diagnosis is what v3 acts on: filter instance-narrow solutions at *admission*, before any gradient is spent (Sec. III-B), keeping the gate for drift.

The multi-seed v3 verdict: an informative negative result. The seeded v3 ladder was run at seeds 42/43/44 (Table IV; rails: a single stream hash and clean canaries on all seeds). The pre-registered success criterion — SCCL_N reduces forgetting vs. the in-run SCCL reference without losing frontier — is *not met*, and the failure is robust rather than noisy: SCCL_N's forgetting is higher on *all three seeds* ($0.131/0.125/0.131$ vs. $0.075/0.075/0.125$; mean 0.129 ± 0.004, the tightest estimate in the table, stably worse on the stability metric), while frontier ties within noise ($+0.521 \pm 0.042$ vs. $+0.504 \pm 0.106$). The admission filter does buy something real: the highest mean accuracy (0.650 ± 0.038, also the lowest accuracy variance of any learning row) and the best backward transfer ($+0.170 \pm 0.038$), consistent with the seed-42 post-hoc gold-telemetry audit showing it raises admitted-target quality ($0.617 \rightarrow 0.667$ mean gold pass rate) at roughly half the plasticity. But better targets do not buy stability on this stream. Per family, the arithmetic holdout collapses on *every* learning row on seeds 42/43 (≤ 0.40 vs. frozen 0.60) and on three of four learning rows on seed 44 (0.15–0.20), with SCCL_N protecting it on *no* seed. Forgetting here is not dominated by target fragility; it is dominated by weight drift *inside accepted updates*, which no accept/reject decision can prevent. Gates protect certified skills; they cannot protect uncertified general capability. That diagnosis is what v4 acts on (Sec. III-C).

TABLE V

SCCL V4 — IN-UPDATE BASE ANCHORING, *as first implemented* (SEED 42, SAME STREAM HASH). **INSTRUMENTATION DISCLOSURE:** A PEFT STATE-DICT KEY MISMATCH MADE THE ANCHOR PENALTY MATCH NO PARAMETER, SO IT EVALUATED TO EXACTLY ZERO; THE TWO ANCHOR ROWS ARE PLAIN-SCCL RUNS IN DISGUISE. THEY ARE BIT-IDENTICAL — NOT MERELY CLOSE: IDENTICAL RAW FLOATS ON EVERY METRIC — TO THE SEED-MATCHED SCCL CONTROLS FROM THE V3 SEED LADDER (λ=0.1 ROW $\equiv$ SEED-44 CONTROL, ACC 0.6875, 16/0; λ=0.5 ROW $\equiv$ SEED-45 CONTROL, ACC 0.525, 14/4), WHICH IS HOW THE FAILURE WAS DETECTED. READ AS SEED CONTROLS, THE LADDER IS THE TIGHTEST VARIANCE MEASUREMENT OF SCCL ON THIS STREAM (ROWS 2–4: SEEDS 42/43/44). THE SCCL AND VSR_NOGOLD ROWS BIT-REPRODUCE THE V3 SEED-42 VALUES ($|\Delta| = 0.000$).

Learner	ACC↑	BWT↑	Forget↓	Frontier↑
Frozen	0.613	+0.132	0.025	+0.588
SCCL (v1 ref), seed 42	0.575	+0.095	0.075	+0.500
"+ anchor, λ=0.1" (inert $\equiv$ SCCL seed 44)	**0.688**	**+0.207**	0.075	**+0.613**
"+ anchor, λ=0.5" (inert $\equiv$ SCCL seed 45)	0.525	+0.045	0.125	+0.400
VSR-nogold (gold tests)	0.637	+0.157	0.125	+0.512

The v4 verdict: an instrumentation post-mortem, and a variance warning. The pre-registered rule cannot be evaluated on this ladder, because the anchor penalty never engaged. The suspicion arose when the λ=0.1 row landed bit-identical to the seed-matched plain-SCCL control from the v3 seed ladder — identical on every metric including raw-float BWT, which a real regularizer cannot produce. Diagnosis: `get_peft_model_state_dict()` strips the adapter segment from its keys (`lora_A.weight`) while the penalty loop matches `named_parameters()` names (`lora_A.default.weight`); the intersection is empty, so the penalty was identically zero in every anchor run (verified on CPU: 0 of 4 keys match). The per-update config audit (λ logged in the gate record) had passed because it audited *wiring*, not *effect*. Two consequences. First, the original verdict's findings — that a weak anchor is a pure plasticity win (ACC 0.688, zero vetoes) and a strong anchor harmful — are **retracted** as anchor claims; those rows measure plain SCCL at seeds 44 and 45. Second, re-read as seed controls the ladder becomes an inadvertent but clean variance measurement: SCCL's ACC spans $0.575/0.688/0.525$ over seeds $42/43/44$ — a spread of 0.163, larger than every mechanism delta reported in this paper. Single-seed arm comparisons on this stream are uninterpretable; the seed-matched controls that exposed the bug had existed all along in the v3 seed ladder. The fix snapshots the anchor from `named_parameters()` so key spaces agree, and is guarded by two engine-level *engagement* tests — key-set agreement with the penalty loop, and strictly smaller distance-to-init under anchoring at identical seed and data — both of which fail on the pre-fix code. The corrected anchor is evaluated inside v5's seeded factorial

(Sec. III-D), where the non-anchor rows must bit-reproduce this ladder as the determinism check across the fix.

Gold telemetry pins the fix on structure, not strictness. The measurement-only gold probe scored each accepted update post-hoc: 15 of the 17 accepted SCCL updates would have been *rejected* by a gold holdout-ϵ gate (probe pass rates dropping $0.600 \to 0.200$–0.400). Replacing the gold-free gate with a gold one would therefore not have preserved arithmetic — it would have halted learning outright (an 88% rejection rate; cf. the v2 rerun, where a gold gate would have accepted 1–4 of 20–23 updates yet every accepted one improved over frozen). The gold-free gate's residual errors are real but far cheaper than gold's over-eagerness; the productive response is to widen what the gate sees, not to make it stricter. That is v5's stratified coverage (Sec. III-D).

SCCL v5: the corrected factorial — first engaged-anchor measurement

The first v5 flight launched before the anchor fix landed, so its anchor axis was inert (Table V post-mortem) and its factorial reading was retracted as an anchor comparison. We therefore re-ran the identical factorial with the engagement-tested engine, with two validations built into the rerun itself. *(i) Determinism across the fix:* the fix touches only the $\lambda>0$ path, so the four non-anchor learners must bit-reproduce the first flight on every headline metric and per-family holdout; all four do (identical raw floats, verified by script from the run artifacts). *(ii) Engagement audit:* every accepted update of the two anchor learners logs the realized penalty `anchor_pen` > 0 in its trajectory (the exact-zero trail is the no-op signature that fooled v4), while the non-anchor learners log exactly zero — engagement and isolation verified from artifacts, not configuration.

TABLE VI
SCCL V5 — CORRECTED 2×2 FACTORIAL WITH THE *engaged* ANCHOR (SEED 42, SAME STREAM HASH; ANCHOR ROWS ARE THE FIRST REAL MEASUREMENTS OF IN-UPDATE ANCHORING IN THIS PAPER). PER-UPDATE `ANCHOR_PEN` AUDIT: 30/30 ACCEPTED UPDATES OF THE TWO ANCHOR LEARNERS (19/19 AND 11/11) LOG A STRICTLY POSITIVE REALIZED PENALTY, WHILE EVERY ACCEPTED UPDATE OF THE NON-ANCHOR LEARNERS LOGS EXACTLY ZERO.

Learner	ACC↑	BWT↑	Forget↓	Frontier↑	Upd/Rbk	Arith	Math
Frozen	0.613	+0.132	0.025	+0.588	0/0	**0.600**	0.250
SCCL (neither; bit-identical control)	0.575	+0.095	0.075	+0.500	17/3	0.400	0.500
+ stratified coverage	0.700	+0.220	0.125	+0.575	14/4	0.200	**1.000**
+ anchor, λ=0.1	0.688	+0.207	0.075	+0.613	19/1	0.400	0.750
+ **both (candidate composition)**	0.688	+0.207	0.075	**+0.613**	11/7	0.400	0.750
VSR-nogold (gold tests)	0.625	+0.145	0.075	+0.550	4/12	0.400	0.500

Pre-registered reading. Three rules, fixed before observing the engaged-anchor data: **H1 (coverage main effect)** — stratified veto alone lifts the arithmetic holdout over SCCL without costing more than 0.02 frontier; **H2 (anchor main effect)** — the engaged anchor lifts arithmetic over SCCL at comparable accuracy; **BREAKTHROUGH (composition)** — the both-cell reaches arithmetic ≥ 0.55 (closing most of the gap to frozen's 0.600) at frontier $\geq$ SCCL -0.02, in which

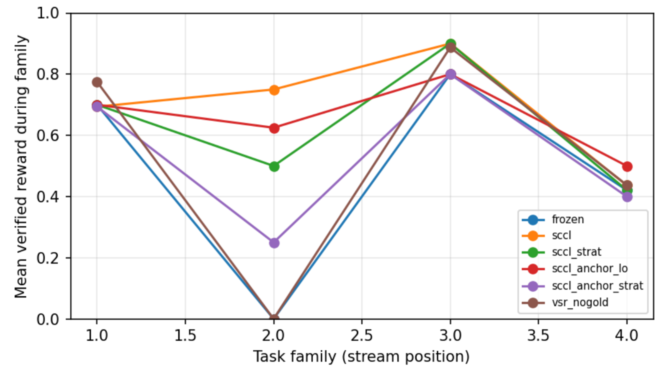

Fig. 1. Mean verified reward per family phase (stream order: arith, math_word, string, drift) for the six corrected-factorial learners, seed 42. Rendered by `gcl/plots.py` from the run's measured trajectories; no hand-edited series.

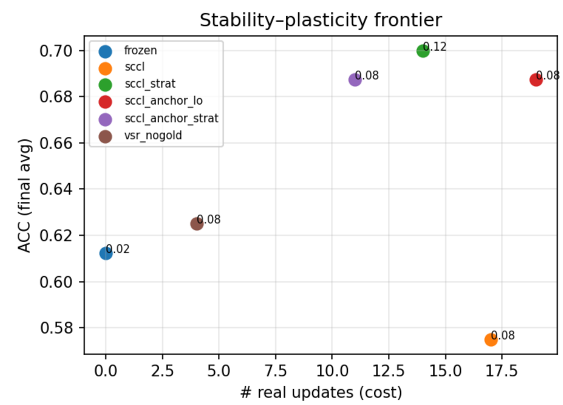

Fig. 2. Stability–plasticity frontier of the corrected factorial: final ACC against the number of real (accepted) updates; annotations give forgetting. The candidate composition is read against the SCCL cell per the pre-registered rules.

case multi-seed confirmation (seeds 43/44, same protocol as v3) precedes any headline claim. Verdict: **all three rules fail on their arithmetic endpoint**, and the failure pattern is itself the finding. H1: stratification *halves* the arithmetic holdout ($0.400 \to 0.200$) despite the stratified pool guaranteeing every family permanent check-pool representation. H2: the engaged anchor leaves arithmetic exactly where SCCL leaves it (0.400) while lifting math ($0.500 \to 0.750$), drift ($0.600 \to 0.800$), and the frontier ($+0.500 \to +0.613$, best in the ladder, with the fewest rollbacks and the lowest forgetting). BREAKTHROUGH: the composition reaches $0.400 < 0.55$ arithmetic; its frontier condition passes ($+0.613 \geq +0.480$), so the rule fails on the endpoint, not on cost.

What the failures share: the gate checks instances, not capabilities. Two independent measurements converge. First, the stratification harm: guaranteeing each family permanent instance-level representation in the check pool did not protect the earliest family — it made erosion *worse*, because updates that break arithmetic *generalization* while preserving the memorized arithmetic instances now pass a pool that always

contains those instances, whereas under the unstratified pool some of them were coincidentally vetoed for also breaking a recent skill. Second, measurement-only gold telemetry on the seed-44 stratified run: 11 of 14 accepted updates degraded the arithmetic gold probe even though every one of them passed the stored self-tests that the gate re-verifies. The gate's own artifacts and the gold telemetry agree on the diagnosis: accepted updates are retaining the stored instances while eroding the capability those instances were evidence for. The anchor, acting inside the update, preserves capability broadly (math, drift, forgetting, BWT all improve) and fully neutralizes the stratification harm to arithmetic ($0.200 \rightarrow 0.400$), but it cannot hold the first family at 0.600 once later-phase training proceeds — its quadratic pull is outrun on the family the stream abandons earliest. The two mechanisms are complementary exactly as designed, and their composition is the best cell in the ladder; the residual is located in the *evidence* the veto consumes. The pre-registered response is to upgrade that evidence: alongside each certified skill, manufacture a held-out *capability probe* — a numeric or paraphrase variant of the spec with fresh self-tests, never trained on — and extend the veto to re-check the newest probe of each family, so an update is admitted only if the capability, not just the instance, survives it (Sec. V).

SCCL v5b: capability probes — upgrading the veto's evidence

The v5 verdict localizes the residual to the evidence the veto consumes: stored instances survive updates that erode the capability they certified. v5b attacks exactly that. For every certified skill the method additionally manufactures one *capability probe*: for arithmetic and math-word families a numeric variant of the spec certified by the same majority vote; for string and drift families a paraphrase variant with freshly derived self-tests (reusing the bidirectional cross-validation of Sec. III-D's neighborhood machinery). Probes are committed to the vault under a distinct kind and are *never trained on*; a bounded pool keeps only the newest probe per family. The RRV veto is extended with one stratum that, before admitting an update, regenerates the newest probe of each family from the current model and requires it to still pass its fresh self-tests (canonical-format numeric match for math domains; assert-line execution for code). The rule is unchanged — any of n samples passing counts as retained — only the evidence is stronger. A pre-registered bounded-damage guard prevents a single noisy probe from stalling learning: if capability vetoes exceed half of a family phase's attempts, the gate arms a margin re-check (two extra regenerations) before vetoing. All switches default off; the frozen and SCCL rows of the v5b ladder must bit-reproduce the v5 rows at the same seeds as the inertness and determinism check. The pre-registered success rule is unchanged in form: the capability-probe stratified cell must reach arithmetic ≥ 0.55 at frontier $\geq$ SCCL -0.02, with multi-seed confirmation preceding any headline claim. The pre-registered prediction: if instance-retention-with-capability-loss is the dominant failure, capability vetoes will fire *inside* family phases and the arithmetic holdout will stabilize near the frozen 0.600 at a measurable plasticity cost. Results (Table VII): the capability probes are the strongest aggregate mechanism in this study so far, and they still fail the arithmetic endpoint — and the failure mode is itself the next finding. The stratified capability cell is the best learner in the ladder on every aggregate measure: accuracy 0.694, above even the frozen baseline (0.613) — genuine absolute learning, not merely damage limitation; best backward transfer ($+0.214$); frontier $+0.562$, second only to the frozen model's free $+0.588$; and a *perfect* math-word holdout (1.000, up from 0.500 under plain SCCL — the largest single-family gain any mechanism has produced), with drift restored to the frozen level (0.800). Engagement was total: $15/15$ probes committed, and every one of the 24 post-certification gates re-checked the newest probe of each family; the frozen and SCCL rows bit-reproduce the v5 rows at the same seeds. Yet arithmetic lands at 0.375 under *both* probe rows — the identical endpoint from two very different gate regimes (9 vs 15 accepted updates) — so H1 fails on the arithmetic endpoint, H2 passes (stratification adds everything else: math $0.750 \rightarrow 1.000$, frontier $+0.494 \rightarrow +0.562$), and the breakthrough rule fails at $0.375 < 0.55$.

TABLE VII
SCCL V5B (SEED 42). CAPABILITY PROBES COMPOSE INTO THE BEST AGGREGATE LEARNER IN THE STUDY (ACC 0.694 BEATS THE FROZEN BASELINE; MATH REACHES 1.000) WHILE THE FIRST FAMILY'S HOLDOUT REMAINS UNPROTECTED AT 0.375 UNDER BOTH PROBE ROWS. DETERMINISM CHECK: FROZEN AND SCCL ROWS BIT-IDENTICAL TO THE V5 CORRECTED FACTORIAL AT THE SAME SEEDS.

learner	ACC	BWT	Forget	Front.	Upd	arith	math
frozen	0.613	+0.132	0.025	+0.588	0	0.600	0.250
SCCL	0.575	+0.095	0.075	+0.500	17	0.400	0.500
SCCL + cap	0.625	+0.145	0.131	+0.494	9	0.375	0.750
SCCL + cap + strat	0.694	+0.214	0.131	+0.562	15	0.375	1.000

Why the probe passes while capability erodes. The arithmetic probe existed from the first family phase and was re-verified at every later gate; every accepted update passed it; the arithmetic holdout still fell from 0.600 to 0.375. One held-out variant per family is too thin a witness: it is satisfied by updates that preserve that variant while eroding the broader capability (the pre-registered branch-(c) diagnosis). A second, unanticipated mechanism compounds it: under high veto pressure the accepted updates are *selected* to fit the current instance and the probe tightly — the capability row's *trained* arithmetic score rises to 0.800 while its arithmetic holdout falls to 0.375, a generalization gap of 0.425 versus 0.200 under plain SCCL. The gate is a filter, not a regularizer: filtering reshapes the accepted-update distribution toward instance-narrow survivors, and the string family pays the starvation cost ($0.800 \rightarrow 0.600$, forgetting 0.131 vs 0.075). Capability-level evidence is confirmed as the right direction — it converted the v5 instance gap into the strongest aggregate result in the study — but the witness must be both *wider* (an ensemble of diverse variants per family rather than one) and paired with

a mechanism that makes surviving updates generalize rather than merely survive.

VI. Discussion

SCCL reframes the supervision problem of deployed continual learning: instead of asking "who provides the tests?", it asks "when is the model's own executable interpretation of a spec trustworthy enough to learn from?" Discriminative consensus gives a computable answer, the unanimous rule turns mastered skills into protected memory, and RRV converts that memory into a stability mechanism. The ladder shows each ablation doing its job: unsafe baselines collapse (verification matters), no-consensus degrades (cross-bag agreement matters) — and full gold supervision underperforms gold-free certification. Gold is neither necessary for safe continual learning nor, when injected as reference targets, sufficient for it; the productive role left for gold is verifying the model's *own* solutions (VSR_NOGOLD), which SCCL's self-tests largely reproduce for free — on accuracy the gold-free match is within run-to-run variance, and the residual frontier gap is quantified above.

The v2/v3/v4 arc is a lesson about *where* stability mechanisms can and cannot act. v1's gate protects trained points; v2 extends protection to a manufactured generalization neighborhood, and measurably buys stability with plasticity (more vetoes, fewer accepted updates, lower accuracy); v3 moves the same neighborhood evidence upstream of the gradient, where a rejection costs a certification attempt instead of a weight update — and multi-seed, that admission filter raises accuracy and its stability but *not* forgetting (Table IV). The general lesson: gates decide *which* updates happen; they cannot control what an accepted update does to weights the gate never inspects. Filtering fragile targets is strictly cheaper than vetoing them after training, and the gate remains the right backstop for distribution drift — but unvisited general capability must be defended *inside* the update itself, which is the role of v4's base anchor (Sec. III-C). The v4 ladder adds a methodological lesson to that arc. Its anchor, as first implemented, silently failed to engage — a state-dict key mismatch left the penalty matching no parameter — and the plausible mechanism story that run initially seemed to support (a weak anchor as a pure plasticity win) dissolved the moment its rows were compared against the seed-matched controls that had existed all along in the v3 seed ladder (Table V). We treat the retraction as part of the record: an audit trail that checks wiring but not effect can pass while a mechanism is inert, so every mechanism reported here now carries an *engagement test* verifying the effect itself. The post-mortem attributes the arithmetic erosion to the gate's recency window: certified families leave the check pool when the stream moves on, so their erosion is never vetoed. In-update regularization and out-of-gradient coverage attack that residual by different routes and are complementary by construction, which is why v5 tests their composition in a factorial design rather than a fifth sequential variant (Sec. III-D).

A methodological point deserves emphasis. Continual-learning benchmarks for LLMs rarely report seed variance; ours, measured directly, exceeds most published-style mechanism deltas on this stream. We read this as a warning for the area and as a design constraint: mechanisms must either produce effects robust across seeds or be reported with them.

Limitations. Single 3B model, one GPU, one small stream; gold-agreement telemetry uses a small probe; correlated misinterpretation of ambiguous specs remains the fundamental failure mode of any gold-free method; open-decision admission under variant-generation failure is a necessary conservatism whose rate we log per run. Scaling the stream, model size, seed count, and probe is ongoing work.

VII. Related Work

Classical continual learning. Stability mechanisms fall into three families. Regularization methods penalize movement of important parameters — EWC uses a Fisher-based importance estimate [2], Synaptic Intelligence accumulates an online path integral of gradient contributions [3], and MAS ties importance to output sensitivity [4]. Replay methods constrain the gradient against stored past examples, either episodically [7] or through projected gradients over a memory buffer [5], [6]. Architecture-based methods partition or grow capacity per task. SCCL intersects both main families with a twist: its certified rehearsal is replay in which the buffer was curated without gold (every pair is consensus-certified), and v4's base anchor is a regularizer whose anchor point θ_0 and importance weights carry no data-derived signal at all — the target is the model's own initialization, so the penalty is gold-free by construction rather than by bookkeeping.

Continual learning of LLMs. Recent work documents and attacks forgetting under continual fine-tuning of large models [8], [9], including LoRA-based approaches that isolate tasks in orthogonal low-rank subspaces [10] or gate adapter integration [11], benchmarks for the setting [12], and surveys of the area [13]. These methods largely assume each task's supervision is available at training time; the deployment regime we study removes that assumption entirely — no task labels, no gold tests, no gold holdout — and asks whether stability can still be enforced. Empirically our setting reproduces the signature finding of the LLM-forgetting literature (capability loss concentrates in components the stream under-visits) but shows it arises *inside accepted updates*, which accept/reject gating alone cannot prevent.

Self-improvement without external verification. Self-training lineages bootstrap from a model's own outputs: STaR iterates rationalization and fine-tuning [16], self-consistency decodes and votes over reasoning paths [17], self-repair and self-debug recondition on execution feedback [18], and self-improvement filters self-generated instruction data by model-scored preference [19]. All of these retain an external filter — task answers, an oracle, gold executables, or a preference model trained on labels — and process supervision likewise trains verifiers from human annotations [20]. Execution-grounded rewards appear in program-synthesis RL, and safe

deployment typically relies on holdout gates. SCCL is, to our knowledge, the first to replace *both* the target-quality verifier and the safety gate with self-certified, structurally gold-free mechanisms, and to enforce that property with adversarial tests.

REFERENCES

[1] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," arXiv:2106.09685, 2021.

[2] J. Kirkpatrick et al., "Overcoming Catastrophic Forgetting in Neural Networks," *PNAS* 114(13):3521–3526, 2017.

[3] F. Zenke, B. Poole, and S. Ganguli, "Continual Learning Through Synaptic Intelligence," in *Proc. ICML*, 2017.

[4] R. Aljundi, F. Babiloni, M. Elhoseiny, M. Rohrbach, and T. Tuytelaars, "Memory Aware Synapses: Learning What (not) to Forget," in *Proc. ECCV*, 2018.

[5] D. Lopez-Paz and M. Ranzato, "Gradient Episodic Memory for Continual Learning," in *Advances in Neural Information Processing Systems (NIPS)*, 2017.

[6] A. Chaudhry, M. Ranzato, M. Rohrbach, and M. Elhoseiny, "Efficient Lifelong Learning with A-GEM," in *Proc. ICLR*, 2019.

[7] D. Rolnick, A. Ahuja, J. Schwarz, T. Lillicrap, and G. Wayne, "Experience Replay for Continual Learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[8] Y. Luo, Z. Yang, F. Meng, Y. Li, J. Zhou, and Y. Zhang, "An Empirical Study of Catastrophic Forgetting in Large Language Models During Continual Fine-tuning," *IEEE/ACM Trans. Audio, Speech, and Language Processing* 33:3776–3786, 2025, arXiv:2308.08747.

[9] Y. Luo, Z. Yang, X. Bai, F. Meng, J. Zhou, and Y. Zhang, "Investigating Forgetting in Pre-Trained Representations Through Continual Learning," arXiv:2305.05968, 2023.

[10] X. Wang, T. Chen, Q. Ge, H. Xia, R. Bao, R. Zheng, Q. Zhang, T. Gui, and X. Huang, "Orthogonal Subspace Learning for Language Model Continual Learning," in *Findings of EMNLP*, pp. 10658–10671, 2023.

[11] Y.-S. Liang, J.-R. Chen, and W.-J. Li, "Gated Integration of Low-Rank Adaptation for Continual Learning of Large Language Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[12] X. Wang, Y. Zhang, T. Chen, S. Gao, S. Jin, X. Yang, Z. Xi, R. Zheng, Y. Zou, T. Gui, Q. Zhang, and X. Huang, "TRACE: A Comprehensive Benchmark for Continual Learning in Large Language Models," arXiv:2310.06762, 2023.

[13] H. Shi, Z. Xu, H. Wang, W. Qin, W. Wang, Y. Wang, Z. Wang, S. Ebrahimi, and H. Wang, "Continual Learning of Large Language Models: A Comprehensive Survey," *ACM Computing Surveys* 58(5):1–42, 2025.

[14] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.

[15] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program Synthesis with Large Language Models," arXiv:2108.07732, 2021.

[16] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman, "STaR: Bootstrapping Reasoning With Reasoning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[17] X. Wang et al., "Self-Consistency Improves Chain of Thought Reasoning in Language Models," in *Proc. ICLR*, 2023.

[18] L. Chen, M. Rabe, E. Austin, D. Charlier, Y. Minsky, and V. Misra, "Teaching Large Language Models to Self-Debug," in *Proc. ICLR*, 2024.

[19] J. Huang, S. S. Gu, L. Hou, Y. Wu, X. Wang, H. Yu, and J. Han, "Large Language Models Can Self-Improve," in *Proc. EMNLP*, pp. 1051–1068, 2023.

[20] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's Verify Step by Step," in *Proc. ICLR*, 2024, arXiv:2305.20050.

VIII. REPRODUCIBILITY

```
python -m gcl.runner --config configs/sccl_main.json
python -m gcl.report --run runs/sccl_main --out
    paper/results.tex
python -m gcl.runner --config configs/sccl_v2.json
python -m gcl.report --run runs/sccl_v2 --out paper/
    results_v2.tex --prefix vtwo
python -m gcl.runner --config configs/sccl_v3.json
      # seeded (torch_seed=42)
python scripts/run_seeds.py --config configs/sccl_v3
    .json --seeds 42,43,44
python -m gcl.report --aggregate runs/sccl_v3_seeds
    --out paper/results_v3.tex --prefix vthree
python -m gcl.runner --config configs/sccl_v4.json
      # v4 base-anchor ladder (seeded)
python -m gcl.runner --config configs/sccl_v5.json
      # v5 anchor x coverage factorial (seeded)
python -m pytest tests/test_sccl.py   # gold-free
    proof suite
```

Gold-freeness is checked by 66 tests: AST audits of `selfcert.py` (covering the probe-manufacture and neighborhood-check APIs), signature inspection, certification semantics under real sandbox execution, poisoned-gold adversarial environments, bidirectional probe cross-validation, certified-rehearsal mixing, admission-time neighborhood certification (open/closed decision policy), RRV rollback behavior across skills, probes, and math entries, the v4 base anchor (per-learner wiring, ladder isolation, decision-independence, per-learner λ override, config-vs-learner-registry consistency so ladder rows cannot be silently skipped, and — added after the key-mismatch disclosure of Sec. V — two engine-level *effect-engagement* tests: key-set agreement between the anchor snapshot and the penalty loop, and strictly reduced distance-to-init under anchoring), and the v5 family-stratified veto pool (deterministic per-family selection, 2×2 factorial wiring, and back-compat of the default recency path so every pre-v5 row stays bit-identical).